

Les réponses doivent être claires, structurées et sans ambiguïté, avec des raisonnements détaillés. Chaque affirmation doit être justifiée rigoureusement en utilisant les définitions appropriées. Enfin, une attention particulière doit être portée à l'orthographe et à la présentation pour garantir une lecture fluide et une organisation soignée des réponses.

Exercice 1 : Machine de Turing

Pour décaler un mot $w = w_1 \dots w_n$ vers la droite sur un ruban infini, la machine de Turing doit insérer un symbole blanc B au début du mot. Voici une description informelle de la machine :

Alphabet d'entrée : $\Sigma = \{a, b\}$

Alphabet de ruban : $\Gamma = \{a, b, B\}$

État initial : q_0

Fonction de transition :

Depuis q_0 , lire le premier symbole w_1 .

Écrire B , déplacer la tête vers la droite.

Si on lit un a , on passe à q_a , si on lit un b , on passe à q_b .

Si on est dans q_a , écrire a et déplacer la tête vers la droite.

Si on lit un a , on reste à q_a , si on lit un b , on passe à q_b .

Si on est dans q_b , écrire b et déplacer la tête vers la droite.

Si on lit un a , on passe à q_a , si on lit un b , on reste à q_b .

Et à la fin du mot, quand on lit B , si on est dans q_a , écrire a , sinon écrire b et passez à q_f .

Un langage L est décidable s'il existe une machine de Turing qui, pour toute entrée w , s'arrête et accepte si $w \in L$, ou rejette sinon.

Décidabilité

Exemple décidable :

Le langage $L = a^n b^n \mid n \geq 0$. Une machine de Turing peut compter les a et vérifier qu'ils correspondent aux b .

Exemple indécidable : Le problème de l'arrêt (Halting Problem), qui consiste à déterminer si une machine de Turing s'arrête sur une entrée donnée.

Réduction polynomiale

Une réduction polynomiale d'un problème A à un problème B est une transformation calculable en temps polynomial qui convertit toute instance de A en une instance de B , préservant la réponse (oui/non). Formellement, $A \leq_p B$ signifie que A n'est pas significativement plus difficile que B .

NP-complétude et réduction

Réponse : Si A est NP-complet et $B \leq_p A$, alors :

$B \in \mathbf{NP}$ (car la réduction préserve les certificats).

Tous les problèmes de NP se réduisent à A , donc aussi à B par transitivité. Ainsi, B est NP-dur. Si $B \in \mathbf{NP}$, alors B est NP-complet.

Exercice 2 : SATisfaisante

Rappel Un problème **SAT** (Satisfiability Problem) est un problème de décision en informatique théorique qui cherche à déterminer si une formule propositionnelle est satisfaisable, c'est-à-dire s'il existe une valuation (ou une assignation) des variables de la formule qui rend la formule vraie.

Une **formule propositionnelle** est une expression logique construite à partir de variables propositionnelles, d'opérateurs logiques (comme la conjonction $\wedge$, la disjonction $\vee$, la négation $\neg$, etc.) et de parenthèses pour définir la priorité des opérations. Par exemple, $(A \wedge B) \vee (\neg C)$ où A , B et C sont des variables propositionnelles.

Une **valuation** est une attribution de valeurs de vérité (**vrai** ou **faux**) aux variables propositionnelles. Une valuation est **satisfaisante** si la formule devient vraie avec ces assignations.

Résoudre le problème SAT revient donc à déterminer s'il existe au moins une valuation satisfaisante pour une formule donnée.

Énoncé A = On considère l'algorithme suivant pour SAT :

« Pour une formule propositionnelle à n variables φ_n prise en entrée, calculer $v(\varphi_n)$ pour toutes les valuations possibles v . Accepter si l'une des valuations envoie φ_n sur **VRAI**, refuser sinon. »

Cet algorithme requiert clairement un temps exponentiel en fonction de n . Ainsi, SAT est de complexité exponentielle, et donc n'appartient pas à **P**.

Comme $\text{SAT} \in \mathbf{NP}$, il s'ensuit que $\mathbf{NP} \neq \mathbf{P}$.

Complexité exponentielle.

L'algorithme énumère toutes les 2^n valuations possibles pour n variables. Le nombre de valuations croît exponentiellement, d'où un temps d'exécution en $O(2^n)$.

Classes P et NP.

P : Problèmes décidables en temps polynomial par une machine de Turing déterministe.

NP : Problèmes décidables en temps polynomial par une machine de Turing non déterministe, ou dont les solutions peuvent être vérifiées en temps polynomial.

Critique de l'énoncé A : L'énoncé affirme que $\text{SAT} \notin \mathbf{P}$ car il admet un algorithme exponentiel, mais cela ne prouve pas l'inexistence d'un algorithme polynomial. La conjecture $\mathbf{P} \neq \mathbf{NP}$ reste non prouvée.

Exercice 3 : Fonctions mystères

On définit les fonctions suivantes :

- La fonction $f : \mathbb{N} \rightarrow \mathbb{N}$, définie par la relation de récurrence :

$$f(n) = f(n-1) + 2 * n - 1, \quad \text{avec } f(0) = 0. \quad (1)$$

- La fonction $g : \mathbb{N} \rightarrow \mathbb{N}$, définie par la relation de récurrence :

$$g(n) = g(n-1) + 3 * f(n) - 3 * n + 1, \quad \text{avec } g(0) = 0. \quad (2)$$

Pour $f(n) = f(n-1) + 2n - 1$ avec $f(0) = 0$:

En développant la récurrence, on trouve $f(n) = n^2$ (somme des premiers impairs).

Pour $g(n) = g(n-1) + 3f(n) - 3n + 1$ avec $g(0) = 0$:

En substituant $f(n) = n^2$, on obtient :

$g(n) = \sum_{k=1}^n (3k^2 - 3k + 1) = n^3$ (par induction).

Le calcul F_n de f du nombre de multiplication pour le calcul de f est clairement linéaire (une seule multiplication par appel) Donc $O(n)$.

le nombre de multiplication pour le calcul de g est $G_n = G_{n-1} + 1 + F_n + 1$.

Donc $G_n = G_{n-1} + 2 + n$, et $G_0 = 0$ qui est de l'ordre de n^2 .

Donc la complexité de f et g terme de nombre de multiplication est respectivement linéaire et quadratique.

Exercice 4 : Coloriage

Un graphe avec au moins 3 sommets est k -coloriable si on peut colorier ses sommets en utilisant au maximum k couleurs, de sorte que deux sommets adjacents aient des couleurs différentes.

1. Caractérisation de la 2-coloration :

Un graphe est **2-coloriable** (c'est-à-dire **biparti**) si et seulement s'il ne contient pas de **cycle de longueur impaire**. En effet, dans un graphe biparti, les sommets peuvent être partitionnés en deux ensembles tels qu'aucune arête ne relie deux sommets d'un même ensemble. Cela implique que tout cycle doit avoir une longueur paire.

Par exemple, un triangle (noté C_3), qui est un cycle de longueur 3, n'est pas 2-coloriable. Il nécessite au moins 3 couleurs pour que deux sommets adjacents ne soient pas colorés de la même couleur.

2. Vérification d'une 3-coloration :

Le problème de 3-COLORATION consiste à déterminer s'il est possible d'attribuer une couleur parmi trois possibles à chaque sommet d'un graphe $G = (V, E)$, de telle sorte que deux sommets adjacents aient toujours des couleurs différentes.

La vérification d'une solution proposée (i.e., d'une coloration donnée) peut se faire en temps polynomial, selon les étapes suivantes :

- S'assurer que chaque sommet $v \in V$ a bien une couleur dans un ensemble de trois couleurs (par exemple, $\{1, 2, 3\}$).
- Pour chaque arête $(u, v) \in E$, vérifier que les couleurs de u et v sont différentes.

Ces vérifications nécessitent un parcours linéaire des sommets et des arêtes, donc elles sont faisables en temps polynomial.

3. Réduction de 3-SAT vers 3-COLORATION :

On montre que le problème de la 3-coloration est **NP-complet** en construisant une **réduction polynomiale** à partir du problème 3-SAT, qui est un problème canonique de la classe NP-complet.

Soit une formule booléenne ϕ en forme normale conjonctive (FNC), constituée de m clauses, chacune contenant exactement trois littéraux. L'objectif est de construire un graphe G_ϕ tel que ϕ est satisfiable si et seulement si G_ϕ est 3-coloriable.

- **Couleurs de référence** : on introduit trois sommets spéciaux que l'on colorera respectivement avec les couleurs : *Vrai*, *Faux* et *Neutre*. Ces trois sommets forment un triangle, assurant qu'ils auront chacun une couleur différente.
- **Variables** : pour chaque variable x_i , on crée deux sommets : x_i et $\neg x_i$, reliés entre eux par une arête. On les relie aussi au sommet *Neutre*, de façon à forcer qu'ils soient colorés soit *Vrai*, soit *Faux*, mais pas *Neutre*, et de manière opposée (puisque'ils sont reliés).
- **Clauses** : pour chaque clause $(l_1 \vee l_2 \vee l_3)$, on définit une construction en forme de triangle, connecté aux trois littéraux l_1 , l_2 et l_3 qui force au moins un des littéraux à être colorié en *Vrai* pour que le triangle soit correctement colorable. Ainsi chaque sommet est coloriable.
- **Correspondance** : une 3-coloration valide de l'ensemble du graphe (respectant les contraintes imposées par les gadgets) correspond à une valuation des variables qui rend ϕ vraie. Ainsi, on a une équivalence entre la satisfiabilité de la formule et la 3-colorabilité du graphe construit.

Cette réduction est réalisable en temps polynomial par rapport à la taille de la formule, ce qui permet de conclure que le problème de 3-coloration est NP-complet.